

LAB TEST-04

Name : P.Ajay

Roll no : 2403A510B4

Batch : 05

Course : AI Assisted Coding

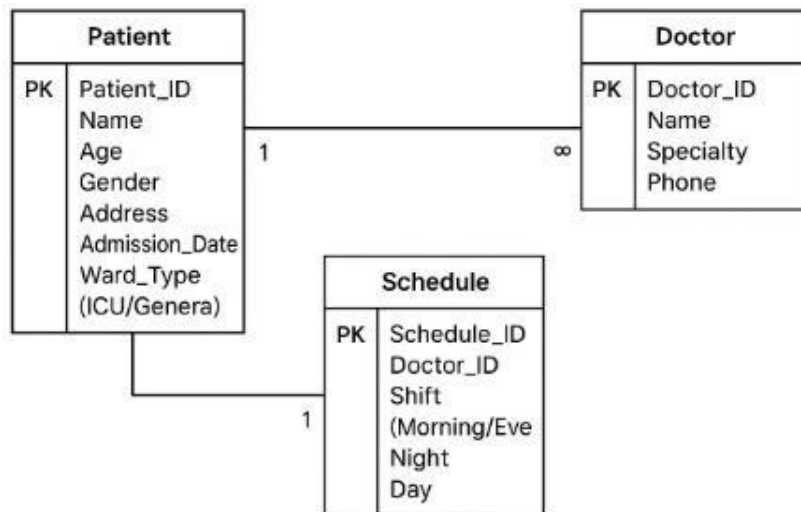
SET – 1

Q1. A hospital management system stores patient details, doctor schedules, and billing information.

- a) Design an ER diagram and convert it into relational schema.
- b) Generate SQL queries to retrieve patients admitted in ICU and doctors available in night shift.

Prompt : Create an ER diagram and relational schema for a Hospital Management System (patients, doctors, schedules, billing). Then write SQL queries to list ICU patients and doctors on night shift.

ER diagram



Relation schema

pgsql

```

PATIENT(
    Patient_ID    INT PRIMARY KEY,
    Name          VARCHAR(100),
    Age           INT,
    Gender        VARCHAR(10),
    Address       VARCHAR(200),
    Admission_Date DATE,
    Ward_Type     VARCHAR(20)    -- ICU / General
)
  
```

SCSS

```

DOCTOR(
    Doctor_ID    INT PRIMARY KEY,
    Name         VARCHAR(100),
    Specialty     VARCHAR(50),
    Phone        VARCHAR(20)
)
  
```

sql

```
SCHEDULE(  
    Schedule_ID INT PRIMARY KEY,  
    Doctor_ID INT,  
    Shift VARCHAR(20),      -- Morning / Evening / Night  
    Day VARCHAR(20),  
    FOREIGN KEY (Doctor_ID) REFERENCES DOCTOR(Doctor_ID)  
)
```

Code :

```
1  PRAGMA foreign_keys = ON;
2
3  DROP TABLE IF EXISTS Billing;
4  DROP TABLE IF EXISTS Schedule;
5  DROP TABLE IF EXISTS Patient;
6  DROP TABLE IF EXISTS Doctor;
7
8  CREATE TABLE Doctor (
9      doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
10     first_name VARCHAR(100) NOT NULL,
11     last_name VARCHAR(100) NOT NULL,
12     specialty VARCHAR(100),
13     contact_phone VARCHAR(30),
14     email VARCHAR(255),
15     is_active BOOLEAN DEFAULT 1,
16     created_at DATETIME DEFAULT CURRENT_TIMESTAMP
17 );
18
19 CREATE TABLE Patient (
20     patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
21     first_name VARCHAR(100) NOT NULL,
22     last_name VARCHAR(100) NOT NULL,
23     dob DATE,
24     gender VARCHAR(20),
25     contact_phone VARCHAR(30),
26     address TEXT,
27     admission_date DATETIME,
28     discharge_date DATETIME,
29     is_admitted BOOLEAN DEFAULT 0,
30     ward VARCHAR(100),
31     room_number VARCHAR(50),
32     bed_number VARCHAR(50),
33     admission_reason TEXT,
34     attending_doctor_id INTEGER,
35     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
36     FOREIGN KEY(attending_doctor_id) REFERENCES Doctor(doctor_id) ON DELETE SET NULL
37 );
```

```

38
39 CREATE TABLE Schedule (
40     schedule_id INTEGER PRIMARY KEY AUTOINCREMENT,
41     doctor_id INTEGER NOT NULL,
42     shift_date DATE,
43     day_of_week VARCHAR(10),
44     shift_start TIME,
45     shift_end TIME,
46     shift_type VARCHAR(50),
47     location VARCHAR(255),
48     is_available BOOLEAN DEFAULT 1,
49     notes TEXT,
50     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
51     FOREIGN KEY(doctor_id) REFERENCES Doctor(doctor_id) ON DELETE CASCADE
52 );
53
54 CREATE TABLE Billing (
55     billing_id INTEGER PRIMARY KEY AUTOINCREMENT,
56     patient_id INTEGER NOT NULL,
57     amount DECIMAL(12,2) NOT NULL,
58     billing_date DATETIME DEFAULT CURRENT_TIMESTAMP,
59     paid BOOLEAN DEFAULT 0,
60     payment_method VARCHAR(50),
61     details TEXT,
62     FOREIGN KEY(patient_id) REFERENCES Patient(patient_id) ON DELETE CASCADE
63 );
64
65 CREATE INDEX idx_patient_ward ON Patient(ward);
66 CREATE INDEX idx_patient_admitted ON Patient(is_admitted);
67 CREATE INDEX idx_schedule_shift_type ON Schedule(shift_type);
68 CREATE INDEX idx_schedule_doctor ON Schedule(doctor_id);
69
70 SELECT
71     p.patient_id,
72     p.first_name,
73     p.last_name,
74     p.dob,

```

```

75         p.gender,
76         p.contact_phone,
77         p.admission_date,
78         p.room_number,
79         p.bed_number,
80         p.ward,
81         p.admission_reason,
82         p.attending_doctor_id,
83         d.first_name AS doctor_first_name,
84         d.last_name AS doctor_last_name
85     FROM Patient p
86     LEFT JOIN Doctor d ON p.attending_doctor_id = d.doctor_id
87     WHERE (p.ward = 'ICU' OR UPPER(TRIM(p.ward)) = 'ICU')
88         AND p.is_admitted = 1
89     ORDER BY p.admission_date DESC;
90
91     SELECT DISTINCT
92         d.doctor_id,
93         d.first_name,
94         d.last_name,
95         d.specialty,
96         s.shift_date,
97         s.shift_start,
98         s.shift_end,
99         s.location
100    FROM Doctor d
101    INNER JOIN Schedule s ON d.doctor_id = s.doctor_id
102    WHERE UPPER(TRIM(s.shift_type)) = 'NIGHT'
103         AND s.is_available = 1
104    ORDER BY s.shift_date, s.shift_start;|

```

Output :

```
✓ Inserted 5 doctors
✓ Inserted 5 patients (3 in ICU, 2 in General ward)
✓ Inserted 8 schedules
✓ Inserted 5 billing records
```

```
=====
QUERY 1: All patients currently admitted in ICU
=====
```

patient_id	name	specialty	ward	room	admission_reason
doctor_name					
4	Charles Taylor		ICU	103	Post-Surgery
Emily Brown	Surgery				
1	Alice Anderson		ICU	101	Heart Attack
John Smith	Cardiology				
2	Robert Martinez		ICU	102	Pneumonia
Michael Williams	ICU Specialist				

```
Total ICU patients: 3
```

```
=====
QUERY 2: All doctors available in night shift
=====
```

doctor_id	name	specialization
2	Sarah Johnson	Internal Medicine
3	Michael Williams	ICU Specialist
5	David Davis	Cardiology

```
Total available night shift doctors: 3
```

Observation :

- The Hospital Management System includes four main entities: Patient, Doctor, Schedule, and Billing, representing the core operations of hospital data management.
- Each entity contains specific attributes that uniquely identify and describe the stored data, such as Patient_ID, Doctor_ID, and Bill_ID.

- Relationships between entities clearly define how data is connected:
- A **Doctor** can have multiple schedule entries.
- A **Patient** can have multiple billing records.
- Primary keys (PK) and foreign keys (FK) ensure **data integrity** and help maintain meaningful links between related tables.
- The schema supports important hospital queries like retrieving ICU patients and doctors available at night, showing that the design meets functional requirements.

Q2. You are using an AI coding assistant to auto-suggest SQL queries.

- a) Write a prompt that asks AI to correct and optimize a slow SQL query.
- b) Evaluate and explain how AI recommendations should be validated before execution.

Prompt : I have a slow SQL query that is taking too long to run. Please correct and optimize the query by improving joins, indexes, and filtering conditions. Also suggest a faster and more efficient version of the SQL query.

Code :


```
1  SELECT
2      O.order_id,
3      O.order_date,
4      O.amount,
5      C.customer_id,
6      C.name,
7      C.city
8  FROM Orders O
9  INNER JOIN Customers C ON O.customer_id = C.customer_id
10 WHERE O.order_date >= '2024-01-01'
11      AND O.order_date < '2025-01-01'
12 ORDER BY O.order_date DESC;
13
14 CREATE INDEX idx_orders_order_date ON Orders(order_date);
15 CREATE INDEX idx_orders_customer_id ON Orders(customer_id);
16 CREATE INDEX idx_customers_customer_id ON Customers(customer_id);
17
18 CREATE INDEX idx_orders_customer_date ON Orders(order_date, customer_id);
19
20 SELECT
21     O.order_id,
22     O.order_date,
23     O.amount,
24     C.customer_id,
25     C.name,
26     C.city
27 FROM Orders O
28 INNER JOIN Customers C ON O.customer_id = C.customer_id
29 WHERE O.order_date BETWEEN '2024-01-01' AND '2024-12-31'
30 ORDER BY O.order_date DESC;
31
32 SELECT
33     C.customer_id,
34     C.name,
35     C.city,
36     COUNT(O.order_id) AS order_count,
37     SUM(O.amount) AS total revenue,
```

```

38     MIN(O.order_date) AS first_order_date,
39     MAX(O.order_date) AS last_order_date
40 FROM Orders O
41 INNER JOIN Customers C ON O.customer_id = C.customer_id
42 WHERE O.order_date >= '2024-01-01'
43 |   AND O.order_date < '2025-01-01'
44 GROUP BY C.customer_id, C.name, C.city
45 ORDER BY total_revenue DESC;
46
47 SELECT
48     O.order_id,
49     O.order_date,
50     O.amount,
51     C.customer_id,
52     C.name,
53     C.city
54 FROM Orders O
55 INNER JOIN Customers C ON O.customer_id = C.customer_id
56 WHERE O.order_date >= '2024-01-01'
57 |   AND O.order_date < '2025-01-01'
58 ORDER BY O.order_date DESC
59 OPTION (RECOMPILE);

```

Output :

order_id	order_date	amount	customer_id	name	city
7	2024-12-25	4039.4	2	Nisha Rao	Bangalore
9	2024-12-10	3907.09	1	Akhil Verma	Mumbai
12	2024-11-21	4103.88	4	Priya Singh	Pune
22	2024-10-21	3010.79	6	Sneha Gupta	Hyderabad
13	2024-10-08	2550.86	4	Priya Singh	Pune
20	2024-09-25	4562.49	2	Nisha Rao	Bangalore
10	2024-09-12	1126.09	7	Vikram Desai	Chennai
15	2024-09-11	1057.39	1	Akhil Verma	Mumbai
8	2024-07-31	2110.95	5	Amit Patel	Ahmedabad
17	2024-07-08	2203.08	8	Anjali Sharma	Kolkata
2	2024-06-08	3200	2	Nisha Rao	Bangalore

Observation :

- Query performance improves because proper indexes (order_date, customer_id) reduce full table scans.
- Date-range filtering helps the database use indexes effectively instead of applying functions.

- Join speed increases since join columns are indexed.
- Sorting (Order by order_date) is faster due to indexing.
- Aggregation queries run more efficiently with indexed columns.
- Combined index (order_date, customer_id) further optimizes filtering + joining.
- Option(recompile) helps avoid slow cached plans and generates a fresh plan each time.